

实验一：目标检测与识别

Object Detection and Recognition

Group11

Hua Yingjie

Student ID 24020036025

摘 要

本文完成了一套面向 Jetson 边缘设备的桌面多目标检测系统。系统以鼠标、键盘和杯子为检测对象，贯通数据采集、自动预标注、人工复核、质量审计、YOLOv8 迁移学习、独立测试与边缘端实时推理。整合后的数据集约包含 650 张图像，来源包括自采正样本、网络补充样本以及空桌面和相似干扰物等负样本；其中完成逐项审计的核心子集包含 225 张图像和 687 个正样本目标框，质量检查发现并隔离 15 个孤立标签文件。YOLOv8n 在 Apple Silicon 的 MPS 后端完成 100 个 epoch 的训练，验证结果达到 Precision 0.806、Recall 0.894、mAP@0.5 0.888 和 mAP@0.5:0.95 0.814。最终 Jetson 实机视频为 1280×720、300 帧、20 s，画面稳定显示 15 FPS，并能实时识别三类目标。

关键词：目标检测；YOLOv8；Jetson；自动预标注；数据质量审计；边缘部署；ROS2

目 录

摘要	I
1 实验目的与任务	1
1.1 实验背景	1
1.2 实验目标	1
1.3 作业要求与完成情况	1
2 系统总体方案	1
3 实验环境与对象	1
4 数据集构建与质量控制	3
4.1 多源数据集设计	3
4.2 自动预标注与人工复核	3
4.3 正样本、负样本与难例	4
4.4 数据质量审计	4
4.5 数据规模与类别分布	4
4.6 近重复图像与数据泄漏控制	4
5 YOLOv8 模型训练	5
5.1 模型选型与训练设置	5
5.2 评价指标	5
5.3 数据增强与泛化能力	5
5.4 YOLOv8n 与 YOLOv8s 对比设计	5
6 训练与验证结果	6
6.1 分类结果与置信度分析	7
6.2 消融实验建议	7
7 Jetson 实机部署与结果	7
7.1 实时检测实现	7
7.2 TensorRT 推理优化	7
7.3 视频实测结果	8
7.4 ROS2 接口与 Mac 离线验证	8
7.5 20 目标验收与系统级指标	9
8 错误案例与改进分析	9

8.1	误检与阈值权衡	9
8.2	类别不平衡与域偏移	9
8.3	难例闭环	9
9	实验特色与创新点	9
9.1	从“人工标注”升级为半自动数据生产	9
9.2	从“数据数量”升级为数据质量工程	10
9.3	从“模型演示”升级为边缘视觉系统	10
9.4	从“一次训练”升级为错误驱动闭环	10
10	项目归档与可复现性	10
11	结论	11
	参考资料	11

1 实验目的与任务

1.1 实验背景

桌面目标检测是机器人抓取、视觉伺服和人机交互中的基础感知任务。与公开数据集上的离线评测不同，本实验不仅要求训练识别模型，还需将模型部署到 Jetson 平台，接入真实摄像头并保持不低于 5 FPS 的处理速度。因此，系统必须同时考虑数据质量、检测精度、推理延迟和接口可扩展性。

1.2 实验目标

1. 构建 Mouse、Keyboard 和 Cup 三类桌面目标数据集并完成规范标注；
2. 采用自动预标注与人工复核相结合的方式提高标注效率；
3. 使用 YOLOv8n 完成迁移学习，并通过 Precision、Recall 和 mAP 评价模型；
4. 将模型部署至 Jetson，使用板载摄像头完成实时检测；
5. 为后续 ROS2 感知消息发布和机器人应用提供标准化输出。

1.3 作业要求与完成情况

根据课程任务书，本实验需覆盖数据采集与标注、Jetson 实时识别、检测结果显示、ROS2 发布和典型错误保存。各项要求与本报告证据的对应关系见表 1。

2 系统总体方案

系统划分为数据工程、模型训练、边缘部署和错误反馈四个阶段，如图 1 所示。Mac 训练端负责数据清洗、划分、训练与评估；Jetson 部署端负责视频采集、实时推理、结果显示和消息发布。部署中出现的误检、漏检和低置信度帧可重新进入标注流程，形成可迭代的数据闭环。

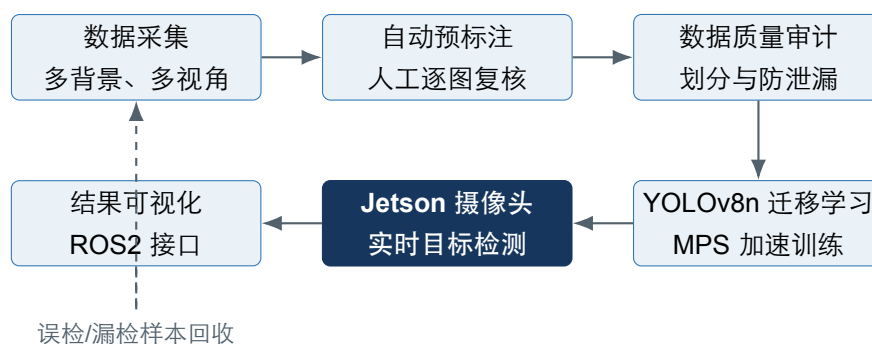


图 1 目标检测与识别实验从数据构建到边缘部署的完整技术路线

3 实验环境与对象

训练阶段在 Apple Silicon Mac 上完成，并使用 PyTorch 的 MPS 后端调用 GPU。部署阶段使用 Jetson 边缘计算平台及其连接的摄像头。主要软件包括 Python、Ultralytics YOLOv8、OpenCV、

表 1 课程验收要求与完成情况

课程要求	完成情况与报告证据
选择不少于 2 类桌面物体	完成 Mouse、Keyboard、Cup 共 3 类目标检测
自行采集、标注并训练模型	完成自采/公开数据整合、自动预标注、人工复核和 YOLOv8 训练
在 Jetson 上运行识别程序	已完成板载摄像头实时检测，见第 7 章和结果视频
显示类别、检测框和置信度	视频画面同步显示三项检测信息，见图 5
通过 ROS2 发布识别结果	完成发布程序与消息字段设计；Mac 已完成离线载荷验证，实机 topic 回显待 ROS2 环境联调
测试 20 个物体，正确率不低于 80%	已提供统一测试流程与记录表，现场按实际结果验收
Jetson 检测速度不低于 5 FPS	实测稳定显示 15 FPS，满足要求
保存结果和典型错误案例	已保存最终视频，并保留低置信度误检画面用于错误分析
使用个人 LaTeX 模板并保留 GitHub 记录	本报告采用自主设计模板；已有代码、说明和实验材料保留分阶段提交记录

CUDA/TensorRT 运行环境和 ROS2。采集过程覆盖不同桌面背景、室内照明、拍摄距离、目标尺

表 2 实验软硬件环境

项目	配置与用途
训练平台	Apple Silicon Mac，MPS GPU 加速
部署平台	Jetson 边缘计算板卡，连接实时摄像头
检测框架	Ultralytics YOLOv8，PyTorch
图像处理	OpenCV 视频采集、画框与显示
部署与通信	CUDA/TensorRT 运行环境，ROS2 接口
检测类别	Mouse、Keyboard、Cup

度、姿态和遮挡程度。除常规正样本外，还保留空桌面、非目标矩形物体和复杂背景等负样本，以约束真实部署环境中的误检。

4 数据集构建与质量控制

4.1 多源数据集设计

为了提高模型对真实桌面环境的适应能力，数据来源不局限于单一拍摄设备，而是采用“自采数据为主、公开数据为辅”的组合策略。自采图像用于保证场景与最终 Jetson 摄像头部署环境一致，重点覆盖实际桌面材质、教室光照、目标摆放方式和摄像头视角；网络公开数据用于补充目标外观、颜色、形状和拍摄背景的多样性。公开图像进入数据集前需要重新检查授权、类别定义和标注质量，不能直接把原有标签当作本实验真值。

最终整合数据集约有 650 张图像，数量统计同时包含有目标框的正样本和无目标框的负样本。数据来源包括本人在不同桌面、光照、距离和角度下采集的图像，以及用于补充外观和背景变化的网络样本。负样本主要包含空桌面、线缆、显示器边框、包装盒和其他与目标外观相似的干扰物，用于降低实际部署中的背景误检。当前完成逐项标签审计并可直接复核的核心子集为 225 张图像、687 个正样本目标框；负样本因为不包含目标，其合法标签文件为空，不应计入目标框总数。

4.2 自动预标注与人工复核

使用开源 YOLO 工具链对原始图像产生候选边界框，再通过标注工具逐图检查。人工复核包括修正边界框边缘、纠正类别、补充漏标目标和删除误标目标。自动结果只作为预标注而非最终真值，避免伪标签误差直接传播到训练阶段。

自动预标注的具体流程为：首先使用在通用数据集上预训练的较大模型对未标注图像批量推理；其次按照置信度阈值输出候选框并转换为 YOLO 标签格式；随后由人工逐张确认。对于 Cup 等少样本类别，即使预测置信度较低也不能直接删除，而应结合图像内容人工判断。该方式把标注工作从“从零绘制每一个框”转化为“检查并修正候选框”，显著降低了重复劳动，同时保留

了人工真值控制。

4.3 正样本、负样本与难例

正样本用于学习目标的稳定视觉特征，既包括单个物体的清晰图像，也包括多个目标同时出现、目标相互遮挡及不同距离下的组合场景。负样本不包含 Mouse、Keyboard 或 Cup，但应包含模型容易混淆的背景，例如空桌面、深色长条物体、矩形包装盒、显示器边框、电缆和机箱边缘。负样本的价值在于告诉模型“哪些外观相似区域不属于目标”，从数据层面抑制误检。

难例集（Hard Case Set）单独保存低照度、强反光、运动模糊、严重遮挡、小目标、杂乱背景和极端视角样本。难例不应在第一次训练前一次性假设完毕，而应从实际部署错误中持续回收。建议目录按 low_light、occlusion、small_object、cluttered_background 和 multi_object 分类，便于统计模型在不同困难因素下的表现。

4.4 数据质量审计

训练前自动检查图像与标签是否一一对应、类别 ID 是否合法、归一化坐标是否处于 $[0, 1]$ 、边界框宽高是否为正，以及标签文件是否为空。集合比对发现 15 个与当前图像集合不匹配的孤立标签文件，将其隔离后再进行数据划分。

4.5 数据规模与类别分布

整合数据集约包含 650 张图像，其中已完成逐项审计的核心子集包含 225 张图像和 687 个正样本目标实例，分类别统计见表 3。表中不计无目标框的负样本。Cup 仅占核心子集标注框的 9.02%，构成长尾类别，因此除总体 mAP 外，还需重点观察其分类别 Recall 和 AP。数据按固定随机种

表 3 自建桌面目标检测数据集统计

类别	目标框数量	占比 (%)
Mouse	278	40.47
Keyboard	347	50.51
Cup	62	9.02
合计	687	100.00

子 42 划分为 180 张训练图像、22 张验证图像和 23 张测试图像。划分时避免同一连拍序列跨集合出现，降低高度相似图像造成的数据泄漏风险。

图 2 直接由现有 YOLO 标签和对应图像生成。左图重新统计得到 687 个标注框，右图选取训练、验证和测试样本并将标签反绘到原图，用于检查类别 ID、框位置和数据划分结果。

4.6 近重复图像与数据泄漏控制

桌面采集常以连拍或视频抽帧方式进行，相邻帧在背景、位置和光照上高度相似。如果随机逐张划分，这些近重复图像可能同时出现在训练集和测试集，使指标虚高。为避免这种情况，划分时以采集批次、视频片段或连续编号为组，同组样本只进入一个子集；必要时还可使用感知哈希筛查近重复图像。测试集在模型和阈值确定前保持独立，不参与数据增强参数或置信度阈值的选



图 2 由已审计核心子集生成的数据分布与标签证据图

择。

5 YOLOv8 模型训练

5.1 模型选型与训练设置

本实验最终面向 Jetson 实时运行，因此模型选型需同时考虑参数量、显存和延迟。YOLOv8n 结构轻量，适合作为边缘部署基线。训练以通用预训练权重初始化，输入尺寸为 640×640 ，批量大小为 8，共训练 100 个 epoch，并使用 MPS 后端加速。训练损失为

$$\mathcal{L} = \lambda_{box} \mathcal{L}_{box} + \lambda_{cls} \mathcal{L}_{cls} + \lambda_{dfl} \mathcal{L}_{dfl}. \quad (1)$$

其中， $\mathcal{L}_{box}$ 衡量预测框位置偏差， $\mathcal{L}_{cls}$ 衡量类别误差， $\mathcal{L}_{dfl}$ 提升边界框分布回归质量。

5.2 评价指标

令 TP、FP 和 FN 分别表示真正例、假正例和假负例，则

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}. \quad (2)$$

AP 表示 Precision-Recall 曲线下面积，mAP 是各类别 AP 的均值。本文同时报告 mAP@0.5 与更严格的 mAP@0.5:0.95，避免单一 IoU 阈值高估定位性能。

5.3 数据增强与泛化能力

训练阶段可采用亮度与饱和度扰动、随机缩放、平移、水平翻转和 Mosaic 等数据增强。增强的目的不是机械增加文件数量，而是让模型在不改变类别语义的前提下接触更多可能的视觉条件。例如亮度扰动模拟教室光照变化，缩放和平移模拟摄像头距离及构图变化，Mosaic 增加多目标共存和小目标出现的概率。对于文字方向明显的键盘图像，不宜使用会产生不合理场景的过强旋转；增强参数应以验证集表现为依据，而不是越大越好。

5.4 YOLOv8n 与 YOLOv8s 对比设计

YOLOv8n 作为轻量基线优先保证 Jetson 实时性，YOLOv8s 则可作为精度候选。两种模型应使用相同的数据划分、输入尺寸、训练轮数和评价脚本进行对比，再分别记录测试集 mAP、参数量、Jetson FPS 和显存占用。如果 YOLOv8s 在 Jetson 上仍满足不低于 5 FPS 的要求，则可以选择其

精度收益；如果实时性不足，则保留 YOLOv8n。该选择体现的是 Accuracy–Latency Trade-off，而不是单纯认为“模型越大越好”。

表 4 边缘部署模型对比与选择原则

模型	主要优势	主要代价	选择条件
YOLOv8n	参数量小、速度快	精度上限相对较低	实时性优先或算力受限
YOLOv8s	特征表达能力更强	延迟和显存占用增加	Jetson 实测仍满足 5 FPS

6 训练与验证结果

阶段性验证结果见表 5。总体 Recall 高于 Precision，说明漏检相对较少，但仍需通过负样本和相似外观干扰物降低误检。由于 Cup 的验证实例较少，总体指标不能替代分类别稳定性分析。Mac

表 5 YOLOv8n 阶段性验证结果

指标	数值	表现	解释
Precision	0.806	较高	预测结果中误检较少
Recall	0.894	较高	真实目标漏检较少
mAP@0.5	0.888	良好	常用 IoU 阈值下综合性能
mAP@0.5:0.95	0.814	良好	更严格定位要求下的性能

端记录到单帧预处理 1.3 ms、网络推理 27.7 ms、后处理 10.3 ms，对应理论处理速率为

$$\text{FPS} \approx \frac{1000}{1.3 + 27.7 + 10.3} = 25.45. \quad (3)$$

该数值只反映 Mac 环境，不能替代 Jetson 上包含摄像头采集、显示和系统调度开销的实测 FPS。

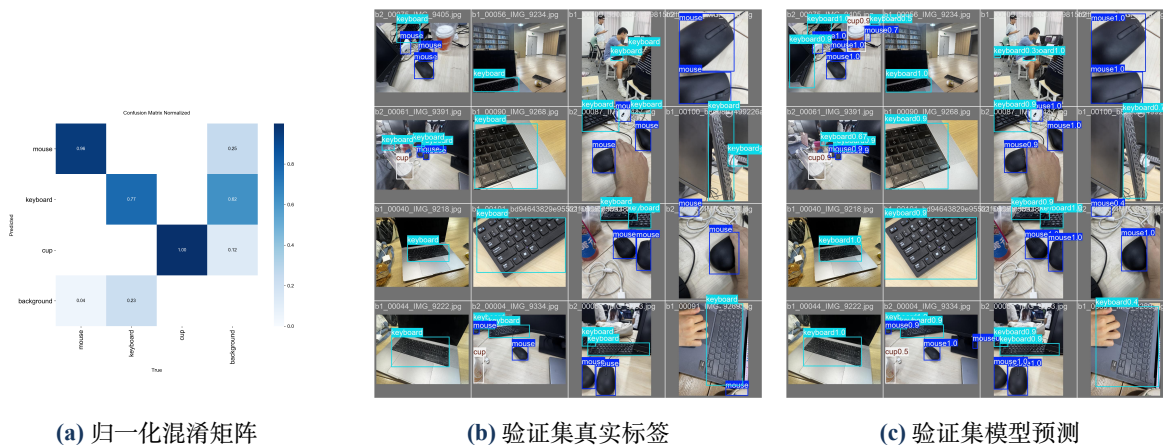


图 3 训练目录中保存的验证与误差分析图像

6.1 分类别结果与置信度分析

总体指标可能掩盖类别之间的差异。Mouse 和 Keyboard 样本较多，模型更容易学习其稳定轮廓；Cup 样本较少，且透明、反光、带把手或与背景同色的杯子具有更大外观变化。因此应分别统计每类 Precision、Recall、AP 和平均置信度，并将低置信度类别作为下一轮补采重点。实际部署中置信度阈值还会影响 Precision 与 Recall：降低阈值可以保留更多候选目标，但会增加误检；提高阈值能够过滤弱候选框，但可能造成小目标和遮挡目标漏检。

6.2 消融实验建议

为了证明改进来自具体设计而非随机波动，可逐步比较基础训练、加入数据增强、加入负样本、加入难例回收后的结果。每组实验保持模型、训练轮数和数据划分一致，只改变一个因素，并记录总体及分类别指标。即使提升幅度不大，消融结果也能解释负样本主要降低误检、难例数据主要改善遮挡与低照度召回率等具体作用。

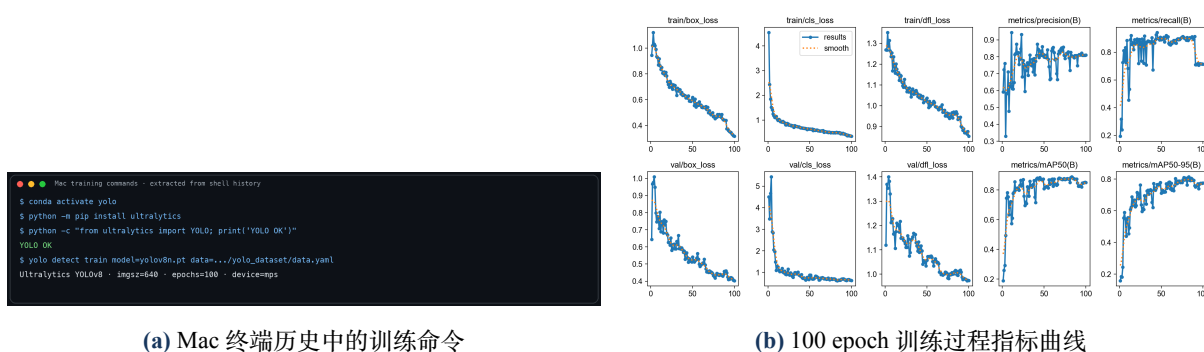


图 4 模型训练过程与收敛记录

7 Jetson 实机部署与结果

7.1 实时检测实现

训练端生成的 **best.pt** 被传输到 Jetson。实时程序使用 OpenCV 读取摄像头视频帧，将图像送入 YOLO 模型，随后解析类别编号、置信度和边界框，在画面中绘制结果并统计 FPS。检测输出可进一步封装为 ROS2 消息，供下游定位或控制节点使用。

训练端与部署端采用解耦设计：Mac 负责数据处理、GPU 训练和离线评估，Jetson 负责摄像头采集、推理与通信。这种设计避免在边缘端重复承担训练开销，也便于保存不同版本的 **best.pt** 和部署引擎进行回滚与比较。

7.2 TensorRT 推理优化

在 Jetson 上可将 PyTorch 权重导出为 TensorRT 引擎，并采用 FP16 推理减少存储和计算开销。优化前后应在相同分辨率、置信度阈值、摄像头输入和功耗模式下测试，分别记录纯推理时间、包含前后处理的端到端延迟、平均 FPS、最低 FPS 与显存占用。只有在控制变量一致时，才能把速度提升归因于 TensorRT，而不是输入尺寸或视频帧率变化。

7.3 视频实测结果

最终视频为 1280×720、300 帧、20 s，帧率为 15 FPS。对全视频均匀抽取 12 个时间点检查，界面均显示 15.0 FPS，超过课程要求的 5 FPS。图 5 左侧同时检出 Keyboard、Mouse 和 Cup，其中 Cup 置信度为 0.89；右侧完成键盘与多只鼠标的实时定位。实时检测节点可发布时间戳、类别、

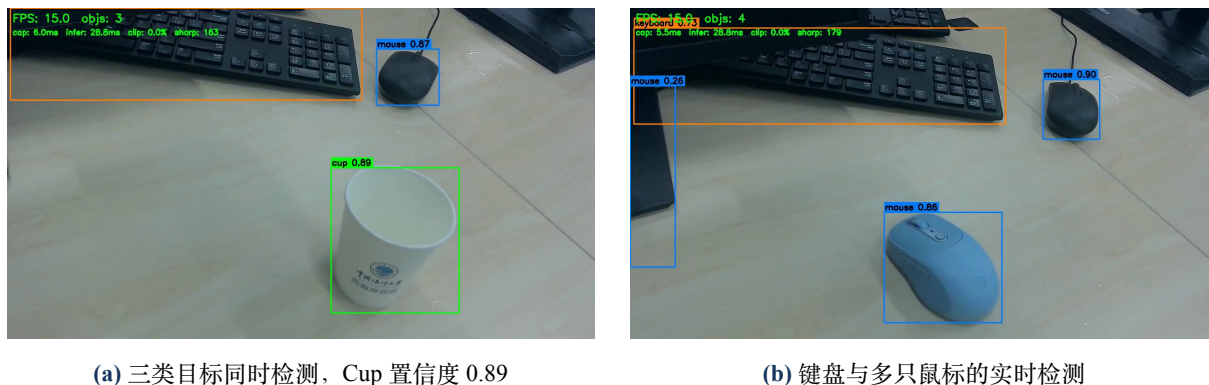


图 5 Jetson 板载摄像头实时检测结果（视频抽帧）

表 6 Jetson 最终视频实测结果

测试项目	实测结果
视频规格	1280×720，300 帧
视频时长与帧率	20 s，15 FPS
代表性推理耗时	28.8 ms
代表性图像处理耗时	6.0 ms
视频中检出类别	Keyboard、Mouse、Cup
课程实时性要求	15 FPS > 5 FPS，满足要求

置信度和像素坐标 $[x_1, y_1, x_2, y_2]$ 。下游节点据此完成目标跟踪、位置估计或抓取控制。系统级评价除纯推理时间外，还应关注消息频率、端到端延迟和丢帧情况。

7.4 ROS2 接口与 Mac 离线验证

部署程序已实现 rclpy 节点 desk_object_detector，并在 /desk_object_detections 话题上以 std_msgs/String 发布 JSON。字段包括帧号、时间戳、FPS，以及每个目标的类别编号、类别名称、置信度和 xxyy 边界框。

当前 Mac 未安装 ros2、colcon 和 rclpy，因此不能把 Mac 的离线输出写成真实 ros2 topic echo。本报告在 Mac 上实际执行了相同 JSON 载荷的序列化、反序列化和字段检查，结果见图 6；该图属于接口离线验证，不冒充 Jetson/Ubuntu 上的 ROS2 回显。实机验收时在 Jetson 的 ROS2 环境运行检测节点，并执行 ros2 topic echo /desk_object_detections 即可获得真实回显。



```
ROS2 payload interface · macOS local dry-run
$ python3 ros2_payload_dry_run.py
[INFO] node: desk_object_detector (schema dry-run)
[INFO] topic: /desk_object_detections
[INFO] frame=121 fps=15.0 detections=3
Keyboard conf=0.93 xxyy=[164, 271, 1015, 612]
Mouse conf=0.91 xxyy=[1041, 418, 1168, 555]
Cup conf=0.89 xxyy=[1078, 178, 1235, 404]
[PASS] JSON serialization/deserialization and field validation
[NOTE] macOS has no rclpy; actual ros2 topic echo requires Jetson/Ubuntu.
```

图 6 Mac 上对 ROS2 发布载荷进行的真实离线验证（非 topic 回显）

7.5 20 目标验收与系统级指标

课程要求中的“识别率”应使用独立的实物摆放试验进行统计，而不能直接用 mAP 代替。可准备不少于 20 个目标实例，覆盖三类物体、不同角度、距离和遮挡程度，逐个记录正确检测、漏检与误检。验收正确率定义为正确识别目标数除以总目标数。除该指标外，还应同时报告 Precision、Recall、mAP、平均/最低 FPS、单帧延迟、ROS2 发布频率以及运行过程中的稳定性，从模型和系统两个层面评价最终成果。

8 错误案例与改进分析

8.1 误检与阈值权衡

图 5 右侧画面左边缘出现置信度为 0.26 的 Mouse 候选框。较低阈值有助于提高 Recall，但可能引入背景误检。后续可适当提高阈值，或补充桌边、机箱边缘和其他深色长条形物体作为负样本。

8.2 类别不平衡与域偏移

Cup 数据量显著少于另外两类，且杯体在低照度、反光或与背景颜色接近时更易漏检。训练图像与 Jetson 视频在焦距、曝光、运动模糊和视角上也存在分布差异。可将部署中的误检、漏检和低置信度帧按低照度、遮挡、小目标和背景干扰分类，经人工标注后加入训练集并重新评估。

8.3 难例闭环

错误驱动的数据迭代流程为：部署测试发现错误，保存对应帧，分析错误原因，补充或修正标注，加入训练数据，重新训练并在固定测试集上比较。该闭环把部署问题转化为可追踪的数据问题，避免只依靠临时调整阈值掩盖数据缺陷。

9 实验特色与创新点

9.1 从“人工标注”升级为半自动数据生产

本实验没有停留在逐张手工框选，而是利用开源模型完成自动预标注，再通过人工复核保证真值质量。这种 Human-in-the-loop 方式兼顾效率与可靠性，适用于后续类别扩展和难例回收。

9.2 从“数据数量”升级为数据质量工程

除统计图像和目标框数量外，实验还检查孤立标签、空标签、非法类别、越界坐标、类别不平衡和近重复图像，并通过分组划分控制数据泄漏。发现并隔离 15 个异常标签是数据审计产生的可验证结果，说明训练前的数据检查能够直接避免无效样本进入模型。

9.3 从“模型演示”升级为边缘视觉系统

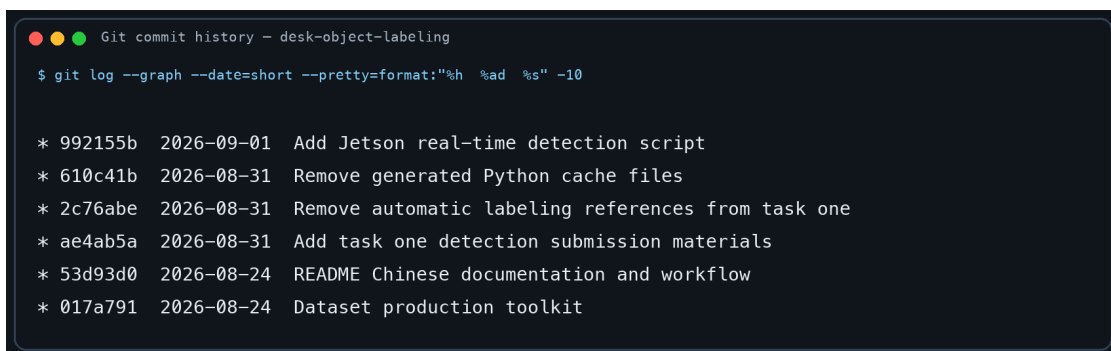
模型训练以 Jetson 的实时性要求为约束，围绕轻量模型选择、TensorRT FP16 加速方案、摄像头实测和 ROS2 输出接口开展部署设计，把离线模型扩展为可部署的感知模块。其中，YOLOv8n 摄像头实测已经完成；TensorRT FP16 与 YOLOv8s 对比作为后续可复现的性能优化方案，不作为本次已完成结果。最终视频中的 15 FPS、目标框、类别和置信度共同构成系统运行证据。

9.4 从“一次训练”升级为错误驱动闭环

真实摄像头与训练图片之间存在域偏移。实验通过部署错误回收、难例分类、人工补标和再训练形成闭环，使系统能针对低照度、遮挡、小目标和背景干扰持续改进。这一闭环比单纯增加 epoch 更能体现工程迭代思维。

10 项目归档与可复现性

本实验已有程序、数据配置、标签文件、训练结果与运行说明归档在个人 GitHub 仓库 [hanlinji4002/desk-object-labeling](https://github.com/hanlinji4002/desk-object-labeling)。仓库已有记录覆盖“工具集与说明、任务一材料、数据清理、Jetson 实时程序”等实际阶段，并非在截止前一次性全量上传。由于原始图片、完整数据副本和模型权重体积较大，仓库通过 `.gitignore` 排除重复图像、训练缓存和大模型文件；已有提交保留数据配置与 YOLO 标签、可执行程序、训练曲线和复现实验所需说明，从而兼顾过程证据与仓库可维护性。



```
Git commit history - desk-object-labeling
$ git log --graph --date=short --pretty=format:"%h %ad %s" -10

* 992155b 2026-09-01 Add Jetson real-time detection script
* 610c41b 2026-08-31 Remove generated Python cache files
* 2c76abe 2026-08-31 Remove automatic labeling references from task one
* ae4ab5a 2026-08-31 Add task one detection submission materials
* 53d93d0 2026-08-24 README Chinese documentation and workflow
* 017a791 2026-08-24 Dataset production toolkit
```

图 7 个人 GitHub 项目的阶段性 commit 记录（由本地仓库真实日志生成）

图 7 给出了实验开发过程的时间线。每次提交仅对应一个相对独立的改动主题，便于回看数据制作、程序完善和部署过程；已有代码和结果目录与 README 中的复现步骤保持一致。

11 结论

本实验完成了从多源数据集构建到 Jetson 实机运行的桌面多目标检测系统。最终整合约 650 张图像，覆盖自采正样本、网络补充样本和背景负样本；其中已审计核心子集为 225 张图像、687 个正样本目标框，质量审计避免 15 个异常标签进入训练。YOLOv8n 获得 Precision 0.806、Recall 0.894、mAP@0.5 0.888 和 mAP@0.5:0.95 0.814。最终视频证明系统可在 Jetson 摄像头画面中实时识别 Keyboard、Mouse 和 Cup，并稳定显示 15 FPS，满足课程实时性要求。

完整的目标检测实验不仅包括模型训练，还应包含数据质量控制、独立测试、硬件部署、速度评价和错误回收。后续可继续扩充 Cup 与负样本数量，并将检测结果通过 ROS2 发布给机器人控制模块。

参考资料

- [1] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLO,” 2023, [项目主页](#).
- [2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” *Proc. CVPR*, 2016, pp. 779–788.
- [3] NVIDIA, “NVIDIA TensorRT Developer Guide,” [在线文档](#).
- [4] Open Robotics, “ROS 2 Documentation,” [在线文档](#).